Optimizing Sink Position in a Wireless Sensor Network [Q4]

Karim Negm

11063520

A. System lifetime with sink at (20,20) → 34.0 minutes

Sensor Coordinates and Sink Position:

- **Define sensor positions**

$$S_i = (x_i, y_i)$$

$$i = 1, 2, \dots, 10$$

Define sink position:

$$Sink = (x_s, y_s) = (20, 20)$$

- **Compute squared Euclidean distance from each sensor to the sink:**

$$d_i^2 = (x_i - x_s)^2 + (y_i - y_s)^2$$

Energy per Packet Transmission:

- **Given packet size b = 2000 bits**
- **Transmission energy per bit:** $E_{tx}(d_i) = k \cdot d_i^2$
  - **where k = 1 nJ/bit/m²**
- **Circuit energy per bit: E_c=50 nJ/bit**
- **Total energy per packet:**

$$E_{per\ packet_i} = \left(E_c + k \cdot d_i^2\right) \cdot b$$

Total Packets per Sensor:

- **Given total available energy per sensor E_b = 5 mJ = 5x10⁶ nJ**

$$N_{packets_i} = \frac{E_b}{E_{per\ packet_i}}$$

Lifetime Calculation:

- **Each sensor transmits one packet every 10 minutes, so the lifetime of sensor is** $L_i = N_{packets_i} \times 10$

System Lifetime:

- **The system lifetime is determined by the first failing sensor** $min(L_i)$

```
%% Q4
% Sensor coordinates (x, y)
sensors = [1 2; 10 3; 4 8; 15 7; 6 1; 9 12; 14 4; 3 10; 7 7; 12 14];

% Part A: Lifetime with Sink at (20,20)
sink_A = [20, 20];
d_sq_A = sum((sensors - sink_A).^2, 2);        % Squared distances
E_per_packet_A = (50 + d_sq_A) * 2000;         % Energy per packet (nJ)
num_packets_A = 5e6 ./ E_per_packet_A;         % Total packets per sensor
lifetimes_A = num_packets_A * 10;              % Lifetime in minutes
system_lifetime_A = min(lifetimes_A);          % System lifetime

fprintf('\nA. System lifetime with sink at (20,20): %.1f minutes\n', system_lifetime_A);
```

B. Optimal sink position: (6.9, 7.7), System lifetime: 214.58 minutes

- Since system lifetime is inversely proportional to d^2 this is effectively a minimax optimization problem where we try to minimize the maximum distance from the sink, MATLAB's optimization toolbox is perfect for this question, and it is what I used to find the optimal coordinate.
- We are trying to minimize a function that takes a sink position (x,y) and outputs the d^2 of all sensors, fminimax heuristically tries many different sink positions trying to minimize the maximum d^2 output of this function.

```
% Part B: Optimal Sink Position
% Objective: Minimize the maximum squared distance to sensors
fun = @(pos) sum((sensors - pos).^2, 2);       % Returns d² for each sensor
initial_guess = mean(sensors);                 % Start with centroid
options = optimoptions('fminimax', 'Display', 'off');
[opt_pos, max_d_sq] = fminimax(fun, initial_guess, [], [], [], [], [], [], [], options);

% Calculate system lifetime at optimal position
E_per_packet_B = (50 + max(max_d_sq)) * 2000;  % Worst-case energy
num_packets_B = 5e6 / E_per_packet_B;          % Packets for worst sensor
system_lifetime_B = num_packets_B * 10;        % System lifetime

fprintf('B. Optimal sink position: (%.1f, %.1f)\n', opt_pos(1), opt_pos(2));
fprintf('   System lifetime: %.2f minutes\n', system_lifetime_B);
```

C. Trade-offs in choosing a fixed sink position versus dynamically moving the sink.

- **Dynamic Positioning Pros:**
    - Could dramatically improve the system lifetime, as the sink can balance out energy consumption in the system, moving closer to the sensors that are more depleted, decreasing the distance and reducing the power consumption in the sensors that are most likely to fail. Thus, less sensors will fail early, no more chain's weakest link effect.
    - In a fixed position system, the nodes that are far-away will fail much earlier than the closer nodes as energy is proportional to distance.

- **Dynamic Positioning Cons:**
    - Requires extra infrastructure to move the sink, and development costs to make it move automatically to the optimal location. Thus, higher system complexity and subsequently costs.
    - Fixed sink position is simple to implement and therefore will cost less.